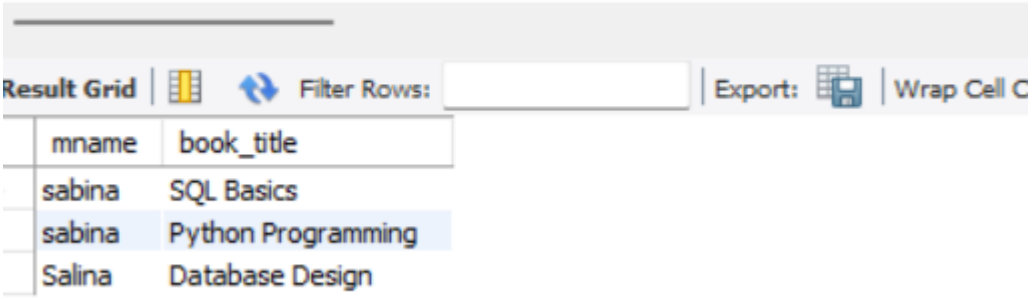
1. INNER JOIN members and loans. Predict the row count before running.

```
SELECT m.name, l.book_title
FROM members m
INNER JOIN loans l
ON m.member_id = l.member_id;
```

```
23 •    SELECT m.mname, l.book_title
24      FROM members AS m
25      INNER JOIN loans AS l
26      ON m.member_id = l.member_id;
27
```



mname	book_title
sabina	SQL Basics
sabina	Python Programming
Salina	Database Design

INNER JOIN returns only members who have matching records in the `loans` table. The row count equals the number of matching loan records.

2. LEFT JOIN showing all members with their books. Who shows NULL and Why?

```
SELECT m.name, l.book_title
FROM members m
LEFT JOIN loans l
ON m.member_id = l.member_id;
```

```
28 •  SELECT m.mname, l.book_title
29    FROM members AS m
30    LEFT JOIN loans AS l
31    ON m.member_id = l.member_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell C

mname	book_title
sabina	SQL Basics
sabina	Python Programming
Salina	Database Design
asmita	NULL

All members appear. Members who have not borrowed any books show `NULL` for `book_title` because they have no matching loan record.

3. Swap the two tables in the LEFT JOIN. Which earlier join does it now behave like?

SELECT m.name, l.book_title
FROM loans l
LEFT JOIN members AS m
ON l.member_id = l.member_id;

```
33 •  SELECT m.mname, l.book_title
34    FROM loans AS l
35    LEFT JOIN members AS m
36    ON m.member_id = l.member_id;
```

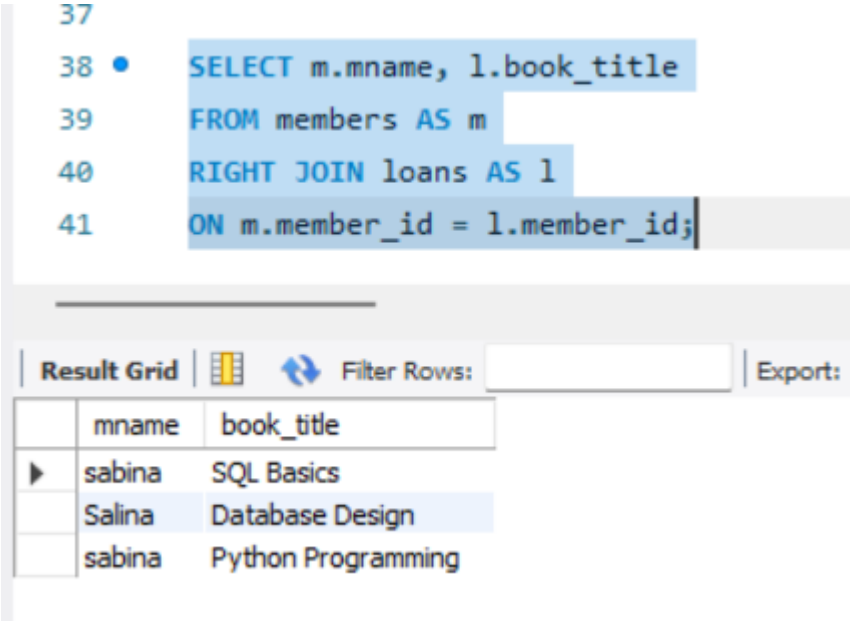
Result Grid | Filter Rows: | Export:

mname	book_title
sabina	SQL Basics
Salina	Database Design
sabina	Python Programming

This behaves like an INNER JOIN when every loan has a valid matching member, because only matching loan records are returned.

4. RIGHT JOIN from members. What appears, and is any row NULL? Why
Not?

SELECT m.name, l.book_title
FROM members m
RIGHT JOIN loans l
ON m.member_id = l.member_id;



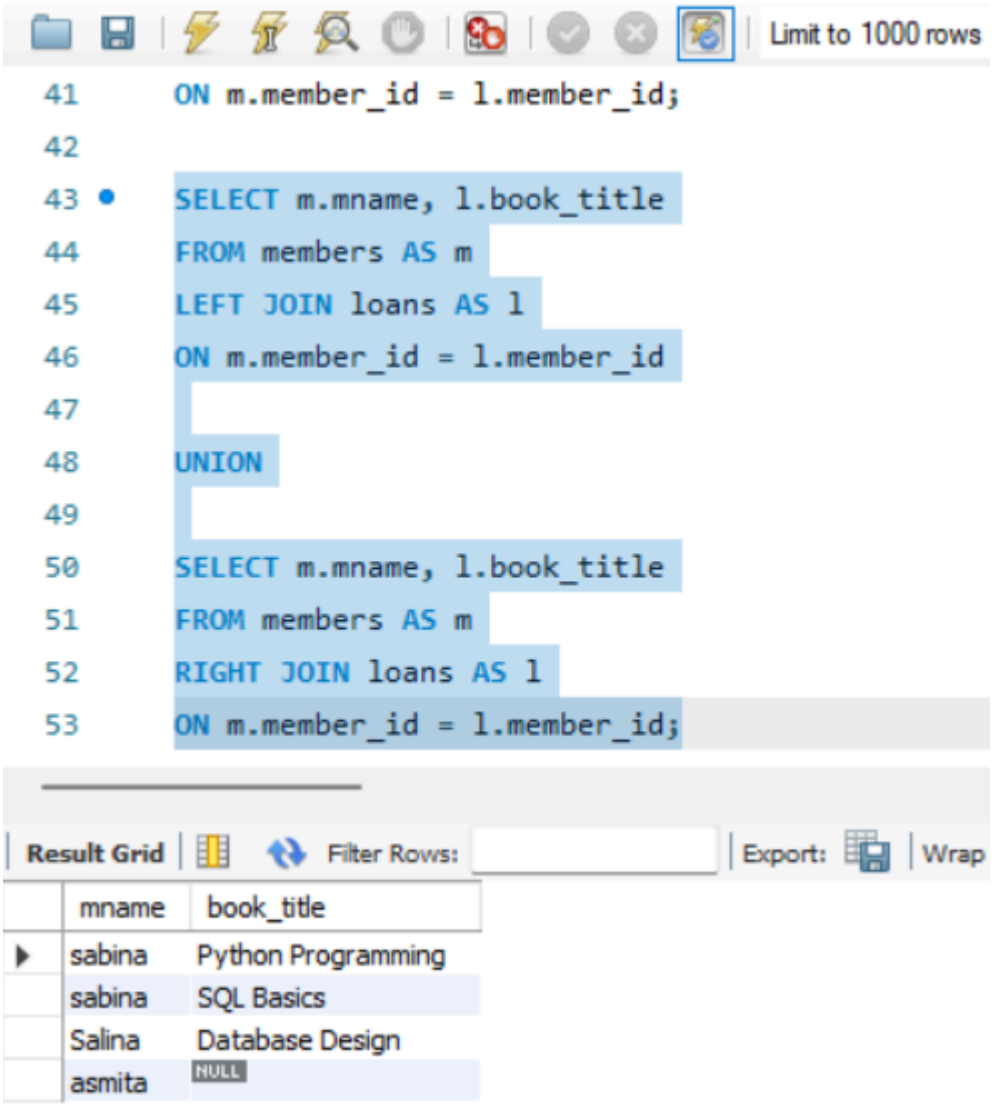
All loans appear, including any loan without a matching member. If every loan has a valid member, no NULL values appear in the member columns.

5. Write the FULL JOIN using UNION. How many rows?

SELECT m.name, l.book_title
FROM members m
LEFT JOIN loans l
ON m.member_id = l.member_id

UNION

SELECT m.name, l.book_title
FROM members m
RIGHT JOIN loans l
ON m.member_id = l.member_id;



This returns matching and non-matching records from both tables. The exact row count depends on your data; `UNION` removes duplicate result rows.

6. Exclusive join: members who borrowed nothing. Which IS NULL check?

```
SELECT m.name
FROM members m
LEFT JOIN loans AS l
ON m.member_id = l.member_id
WHERE l.member_id IS NULL;
```

```
55 •  SELECT m.mname
56    FROM members AS m
57    LEFT JOIN loans AS l
58    ON m.member_id = l.member_id
59    WHERE l.loan_id IS NULL;
```

Result Grid | Filter Rows: | Exp

mname
asmita

`WHERE l.member_id IS NULL` finds members who have no matching loan record.

7. Show member names with book titles only for fines above 50 (join + WHERE).

```
SELECT m.name, l.book_title
FROM members m
INNER JOIN loans l
ON m.member_id = l.member_id
WHERE l.fine > 50;
```

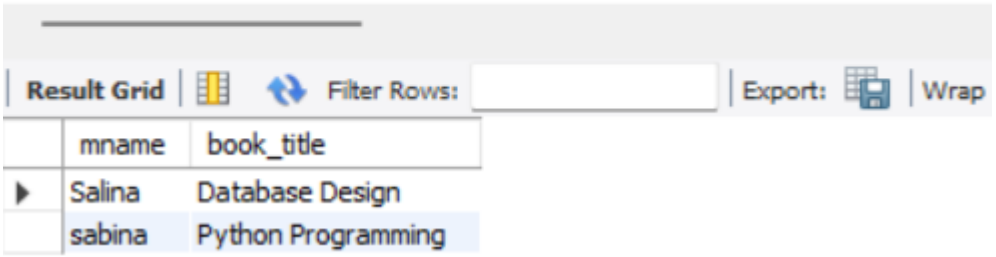
The query displays member names and book titles only for loans where the fine is greater than 50.

8. Delete one loan, run the INNER JOIN again. Which rows disappeared?

```
DELETE FROM loans
WHERE loan_id = 1;
```

```
SELECT m.mname, l.book_title
FROM members AS m
INNER JOIN loans AS l
ON m.member_id = l.member_id;
```

```
73 •  SELECT m.mname, l.book_title
74    FROM members AS m
75    INNER JOIN loans AS l
76    ON m.member_id = l.member_id;
```



mname	book_title
Salina	Database Design
sabina	Python Programming

The deleted loan row disappears from the INNER JOIN result. If that was a member's only loan, that member will no longer appear in the INNER JOIN result.

9. In your own words: when do you choose LEFT over INNER? Give a library Example.

I choose LEFT JOIN when I want to show all records from the left table, even if there is no match in the right table. For example, I use LEFT JOIN to display all library members, including members who have never borrowed a book.

10. Bonus self join: add a referred by column to members (who invited whom) and list inviter with invited.

```
ALTER TABLE members
ADD COLUMN referred_by INT;

ALTER TABLE members
ADD CONSTRAINT fk_referrer
FOREIGN KEY (referred_by)
REFERENCES members(member_id);

SELECT
   inviter.name AS inviter,
   invited.name AS invited_member
FROM members invited
INNER JOIN members inviter
ON invited.referred_by = inviter.member_id;
```

A self JOIN joins the `members` table to itself to show which member invited another member.